

TS-A 1: Merge Sort (Exact Copy)

Platform: Falsicode Code Audit System | Language: Python 3.x (.py) | Max Score: 100 Pts | Scenario: TS-A Baseline

1. Assignment Overview & Scenario

In this activity, you will implement a fully functional **Merge Sort** algorithm from scratch in Python. Merge Sort is an asymptotically optimal, comparison-based, Divide-and-Conquer sorting algorithm. It functions by dividing an unsorted list into n sublists (each of length 1), and repeatedly merging adjacent sublists to produce new sorted sublists until only one completely sorted list remains.

Key Learning Objectives:

- Master the three steps of Divide-and-Conquer: **Divide** (split array), **Conquer** (sort subarrays recursively), and **Combine** (merge sorted lists).
- Implement recursive function calls, base condition formulation, and list slicing/pointer tracking.
- Maintain theoretical complexity bounds: **$O(N \log N)$** Time across all cases and **$O(N)$** Auxiliary Space.
- Produce clean, syntactically valid Python source code suitable for Abstract Syntax Tree (AST) parsing and N-Gram similarity benchmarking.

2. Technical Specifications & Requirements

Your Python solution file must implement the following core functions without utilizing Python's built-in `sorted()` or `.sort()` methods:

Function / Component	Specification & Behavior	Complexity Standard
merge_sort(arr)	Recursively partitions the list <code>arr</code> at midpoint (<code>mid = len(arr) // 2</code>). Recursively sorts the left half (<code>arr[:mid]</code>) and right half (<code>arr[mid:]</code>). Calls <code>merge()</code> to combine halves and returns the sorted list.	Time: $O(N \log N)$ Space: $O(N)$
merge(left, right)	Accepts two pre-sorted lists (<code>left</code> and <code>right</code>). Uses two pointer indices (<code>i, j</code>) to compare front elements and append the smaller element into a new result list. Flushes remaining elements once one list is exhausted.	Time: $O(N)$ Space: $O(N)$
Data Compatibility	Must support standard comparable items: positive/negative integers, floats, and preserve stable sorting for duplicate values.	Deterministic Output
Base Cases & Safety	Must safely return lists with 0 or 1 element immediately (e.g. if <code>len(arr) <= 1</code> : return <code>arr</code>) without error or infinite recursion.	Graceful Termination

3. Example Test Driver & Expected Output

Your code should be verifiable when tested with standard driver code such as the following:

```
# Sample Verification Driver:
from merge_sort import merge_sort

arr1 = [38, 27, 43, 3, 9, 82, 10]
print('Sorted Array: ', merge_sort(arr1))      # Expected: [3, 9, 10, 27, 38, 43, 82]

arr2 = [5, -2, 8, 2, 5, -10, 0]
print('Duplicates & Neg:', merge_sort(arr2))   # Expected: [-10, -2, 0, 2, 2, 5, 5, 8]

arr3 = [1, 2, 3, 4, 5]
print('Already Sorted: ', merge_sort(arr3))    # Expected: [1, 2, 3, 4, 5]

print('Single Element: ', merge_sort([99]))    # Expected: [99]
print('Empty List:      ', merge_sort([]))     # Expected: []
```

4. Submission Guidelines & Constraints

- **File Format:** Submit your solution as a single Python source file (e.g. merge_sort.py).
- **No Built-in Sorting:** Do not use .sort(), sorted(), or external sorting packages. Use standard Python only.
- **Code Cleanliness & Documentation:** Include comments stating time and space complexity for each subroutine.
- **Syntax Validity:** Ensure your code compiles and runs cleanly without syntax errors before uploading.

5. Grading Rubric (100 Points Total)

Criteria	Description & Standards	Points
Core Merge Sort Logic	Accurate recursive partitioning into halves until base case length ≤ 1 .	40 Pts
Two-Way Merge Subroutine	Correct pointer comparisons and sequential list merging in $O(N)$ time.	25 Pts
Edge Case Handling	Handles empty lists, single-element lists, duplicates, and negatives gracefully.	15 Pts
Code Quality & Complexity	Clean structure, readable variable names, PEP 8 formatting, and complexity comments.	20 Pts
Total Score	Full marks achievable for complete, tested, and verified implementation.	100 Pts

Academic Integrity & Research Notice:

This activity is part of the **Falsicode: Code Plagiarism Detection in DSA using AST, N-Grams, and TF-IDF** research study. Submissions will be evaluated using Abstract Syntax Tree (AST) tokenization, normalized N-Gram matching, and TF-IDF similarity vectors. Please ensure you write your own original code.

Prepared by:	Reviewed by:	Checked by:	Approved by:
Faculty Instructor DSA Laboratory Committee	Department Chair Computer Science Dept.	College Dean College of Computing	VP for Academics Academic & Student Services
Date: _____	Date: _____	Date: _____	Date: _____